



Estd. 2000

**ABES Engineering College, Ghaziabad**

Department of Computer Science & Engineering

# **JAVA LAB MANUAL**

**Object Oriented Programming with Java Lab  
(BCS-452)**

Session 2025–26

# Contents

|    |                                                                          |    |
|----|--------------------------------------------------------------------------|----|
| 1  | Matrix Addition Using Two-Dimensional Arrays                             | 3  |
| 2  | Implementation of Variable Size (Jagged) Arrays in Java                  | 4  |
| 3  | Prime Number Verification Using Command Line Arguments                   | 5  |
| 4  | Palindrome Number Check Using Command Line Arguments                     | 6  |
| 5  | Student Class Implementation Using Parameterized Constructor             | 7  |
| 6  | Bank Account Management Using Encapsulation and Menu-Driven Program      | 8  |
| 7  | Demonstration of Compile-Time Polymorphism Using Method Overloading      | 10 |
| 8  | Implementation of Single Inheritance Using Person and Student Classes    | 11 |
| 9  | Demonstration of Runtime Polymorphism Using Method Overriding            | 12 |
| 10 | Implementation of Multiple Inheritance Using Interfaces                  | 13 |
| 11 | User-Defined Exception Handling for Input Validation                     | 14 |
| 12 | Multithreading Implementation Using Even and Odd Threads                 | 15 |
| 13 | Exception Handling Using Multiple Catch Blocks and Finally Block         | 16 |
| 14 | Input Validation Using throw and throws Keywords                         | 17 |
| 15 | Multithreading Implementation Using Runnable Interface                   | 18 |
| 16 | Package Creation and Implementation Using User-Defined Package           | 19 |
| 17 | Implementation of Various Input Methods in Java                          | 20 |
| 18 | Secure Input Handling Using Console Class                                | 22 |
| 19 | Development of RESTful Employee Management System Using Spring Boot      | 23 |
| 20 | Enhanced RESTful Employee CRUD Application Using Spring Boot             | 25 |
| 21 | Development of Frontend-Based Employee Management System Using Thymeleaf | 26 |
| 22 | Implementation of In-Memory CRUD Operations Using REST API               | 28 |

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>23 Book Store Management System Using Spring Boot and In-Memory Storage</b> | <b>29</b> |
| <b>24 Book Store Management System Using Spring Boot, H2 Database, and JPA</b> | <b>31</b> |

## Experiment 1 Matrix Addition Using Two-Dimensional Arrays

### Objective:

To write a Java program that performs the addition of two matrices.

### Source Code:

```
1 // Experiment 1: Matrix Addition
2 import java.util.Scanner;
3
4 public class MatrixAddition {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         int[][] a = new int[2][2];
9         int[][] b = new int[2][2];
10        int[][] sum = new int[2][2];
11
12        System.out.println("Enter elements of first matrix:");
13        for(int i=0;i<2;i++){
14            for(int j=0;j<2;j++){
15                a[i][j] = sc.nextInt();
16            }
17        }
18
19        System.out.println("Enter elements of second matrix:");
20        for(int i=0;i<2;i++){
21            for(int j=0;j<2;j++){
22                b[i][j] = sc.nextInt();
23            }
24        }
25
26        for(int i=0;i<2;i++){
27            for(int j=0;j<2;j++){
28                sum[i][j] = a[i][j] + b[i][j];
29            }
30        }
31
32        System.out.println("Resultant Matrix:");
33        for(int i=0;i<2;i++){
34            for(int j=0;j<2;j++){
35                System.out.print(sum[i][j] + " ");
36            }
37            System.out.println();
38        }
39    }
40 }
```

## Experiment 2 Implementation of Variable Size (Jagged) Arrays in Java

### Objective:

To create and print a variable-sized array (Jagged Array).

### Source Code:

```
1 // Experiment 2: Jagged Array
2 public class JaggedArray {
3     public static void main(String[] args) {
4         int[][] arr = new int[3][];
5
6         arr[0] = new int[2];
7         arr[1] = new int[3];
8         arr[2] = new int[4];
9
10        int count = 1;
11
12        for(int i=0;i<arr.length;i++){
13            for(int j=0;j<arr[i].length;j++){
14                arr[i][j] = count++;
15            }
16        }
17
18        for(int i=0;i<arr.length;i++){
19            for(int j=0;j<arr[i].length;j++){
20                System.out.print(arr[i][j] + " ");
21            }
22            System.out.println();
23        }
24    }
25 }
```

## Experiment 3 Prime Number Verification Using Command Line Arguments

### Objective:

To check whether a number is prime, use command line arguments.

### Source Code:

```
1 // Experiment 3: Prime Check (Command Line)
2 public class PrimeCheck {
3     public static void main(String[] args) {
4         int num = Integer.parseInt(args[0]);
5         int count = 0;
6
7         for(int i=1;i<=num;i++){
8             if(num % i == 0) count++;
9         }
10
11         if(count == 2)
12             System.out.println("Prime Number");
13         else
14             System.out.println("Not Prime");
15     }
16 }
```

## Experiment 4 Palindrome Number Check Using Command Line Arguments

### Objective:

To check whether a number is a palindrome using command line arguments.

### Source Code:

```
1 // Experiment 4: Palindrome Check (Command Line)
2 public class PalindromeCheck {
3     public static void main(String[] args) {
4         int num = Integer.parseInt(args[0]);
5         int original = num;
6         int rev = 0;
7
8         while(num != 0){
9             int digit = num % 10;
10            rev = rev * 10 + digit;
11            num = num / 10;
12        }
13
14        if(original == rev)
15            System.out.println("Palindrome");
16        else
17            System.out.println("Not Palindrome");
18    }
19 }
```

## Experiment 5 Student Class Implementation Using Parameterized Constructor

### Objective:

To create a Student class, initialize details using a parameterized constructor and display student information.

### Source Code:

```
1 // Experiment 5: Student Class with Parameterized Constructor
2 class Student {
3     String name;
4     int roll;
5
6     Student(String n, int r){
7         name = n;
8         roll = r;
9     }
10
11     void display(){
12         System.out.println("Name: " + name);
13         System.out.println("Roll No: " + roll);
14     }
15
16     public static void main(String[] args) {
17         Student s1 = new Student("Aditya", 101);
18         s1.display();
19     }
20 }
```



## Experiment 6 Bank Account Management Using Encapsulation and Menu-Driven Program

### Objective

To demonstrate encapsulation, a BankAccount class will be created with private data members, and operations such as deposit, withdrawal, and balance display will be performed using a menu-driven program.

### Source Code

```
1 import java.util.Scanner;
2
3 class BankAccount {
4     private double balance;
5
6     BankAccount() {
7         balance = 0;
8     }
9
10    public void deposit(double amt) {
11        if (amt > 0) balance += amt;
12        else System.out.println("Invalid deposit amount.");
13    }
14
15    public void withdraw(double amt) {
16        if (amt <= 0) {
17            System.out.println("Invalid withdraw amount.");
18        } else if (amt > balance) {
19            System.out.println("Insufficient Balance.");
20        } else {
21            balance -= amt;
22        }
23    }
24
25    public void displayBalance() {
26        System.out.println("Current Balance: " + balance);
27    }
28 }
29
30 public class Exp6_BankMenu {
31     public static void main(String[] args) {
32         Scanner sc = new Scanner(System.in);
33         BankAccount acc = new BankAccount();
34
35         while (true) {
36             System.out.println("\n1. Deposit\n2. Withdraw\n3.
37                               Balance\n4. Exit");
38             System.out.print("Enter choice: ");
39             int ch = sc.nextInt();
```

```
39
40         if (ch == 1) {
41             System.out.print("Enter amount: ");
42             acc.deposit(sc.nextDouble());
43         } else if (ch == 2) {
44             System.out.print("Enter amount: ");
45             acc.withdraw(sc.nextDouble());
46         } else if (ch == 3) {
47             acc.displayBalance();
48         } else if (ch == 4) {
49             break;
50         } else {
51             System.out.println("Invalid choice.");
52         }
53     }
54 }
55 }
```

## Experiment 7 Demonstration of Compile-Time Polymorphism Using Method Overloading

### Objective

To demonstrate compile-time polymorphism (method overloading) using a Calculator class to add integers, floating-point values, and three numbers.

### Source Code

```
1 class Calculator {  
2     int add(int a, int b) { return a + b; }  
3     double add(double a, double b) { return a + b; }  
4     int add(int a, int b, int c) { return a + b + c; }  
5 }  
6  
7 public class Exp7_Overloading {  
8     public static void main(String[] args) {  
9         Calculator c = new Calculator();  
10        System.out.println(c.add(10, 20));  
11        System.out.println(c.add(10.5, 2.5));  
12        System.out.println(c.add(1, 2, 3));  
13    }  
14 }
```

## Experiment 8 Implementation of Single Inheritance Using Person and Student Classes

### Objective

To demonstrate single inheritance, create a Person class and derive a Student class, then display student details using inherited members.

### Source Code

```
1  class Person {
2      String name;
3      int age;
4
5      Person(String name, int age){
6          this.name = name;
7          this.age = age;
8      }
9  }
10
11 class Student extends Person {
12     int roll;
13
14     Student(String name, int age, int roll){
15         super(name, age);
16         this.roll = roll;
17     }
18
19     void display(){
20         System.out.println("Name: " + name);
21         System.out.println("Age: " + age);
22         System.out.println("Roll: " + roll);
23     }
24 }
25
26 public class Exp8_Inheritance {
27     public static void main(String[] args) {
28         Student s = new Student("Aman", 20, 101);
29         s.display();
30     }
31 }
```

## Experiment 9 Demonstration of Runtime Polymorphism Using Method Overriding

### Objective

To demonstrate runtime polymorphism (method overriding) using Shape as the base class and Circle/Rectangle as derived classes overriding area().

### Source Code

```
1  abstract class Shape {
2      abstract double area();
3  }
4
5  class Circle extends Shape {
6      double r;
7      Circle(double r){ this.r = r; }
8      double area(){ return 3.14159 * r * r; }
9  }
10
11 class Rectangle extends Shape {
12     double l, b;
13     Rectangle(double l, double b){ this.l = l; this.b = b; }
14     double area(){ return l * b; }
15 }
16
17 public class Exp9_Overriding {
18     public static void main(String[] args) {
19         Shape s1 = new Circle(5);
20         Shape s2 = new Rectangle(4, 6);
21
22         System.out.println("Circle Area: " + s1.area());
23         System.out.println("Rectangle Area: " + s2.area());
24     }
25 }
```

## Experiment 10 Implementation of Multiple Inheritance Using Interfaces

### Objective

To justify that Java does not support multiple inheritance using classes, but that multiple inheritance is possible using interfaces.

### Source Code

```
1 interface A {
2     void showA();
3 }
4
5 interface B {
6     void showB();
7 }
8
9 class C implements A, B {
10     public void showA(){ System.out.println("A implemented"); }
11     public void showB(){ System.out.println("B implemented"); }
12 }
13
14 public class Exp10_MultipleInheritanceInterfaces {
15     public static void main(String[] args) {
16         C obj = new C();
17         obj.showA();
18         obj.showB();
19
20         // Note: Multiple inheritance with classes is NOT
21         //         allowed in Java.
22         // class X extends P, Q --> Compilation error
23     }
24 }
```

## Experiment 11 User-Defined Exception Handling for Input Validation

### Objective

To implement user-defined exception handling for negative amounts entered.

### Source Code

```
1 import java.util.Scanner;
2
3 class NegativeAmountException extends Exception {
4     NegativeAmountException(String msg){
5         super(msg);
6     }
7 }
8
9 public class Exp11_UserDefinedException {
10     static void validateAmount(double amt) throws
11         NegativeAmountException {
12         if (amt < 0) throw new NegativeAmountException("
13             Negative amount not allowed!");
14     }
15
16     public static void main(String[] args) {
17         Scanner sc = new Scanner(System.in);
18         System.out.print("Enter amount: ");
19         double amt = sc.nextDouble();
20
21         try {
22             validateAmount(amt);
23             System.out.println("Valid amount: " + amt);
24         } catch (NegativeAmountException e) {
25             System.out.println("Exception: " + e.getMessage());
26         }
27     }
28 }
```

## Experiment 12 Multithreading Implementation Using Even and Odd Threads

### Objective

To create two threads, "Even" and "Odd", that print even numbers every 2 seconds and odd numbers every 5 seconds.

### Source Code

```
1  class EvenThread extends Thread {
2      public void run() {
3          for(int i=2; i<=20; i+=2){
4              System.out.println("Even: " + i);
5              try { Thread.sleep(2000); } catch(Exception e){}
6          }
7      }
8  }
9
10 class OddThread extends Thread {
11     public void run() {
12         for(int i=1; i<=19; i+=2){
13             System.out.println("Odd: " + i);
14             try { Thread.sleep(5000); } catch(Exception e){}
15         }
16     }
17 }
18
19 public class Exp12_EvenOddThreads {
20     public static void main(String[] args) {
21         new EvenThread().start();
22         new OddThread().start();
23     }
24 }
```



## Experiment 13    Exception Handling Using Multiple Catch Blocks and Finally Block

### Objective

To demonstrate exception handling, use multiple catch blocks and a finally block.

### Source Code

```
1 public class Exp13_MultiCatchFinally {
2     public static void main(String[] args) {
3         try {
4             int a = Integer.parseInt(args[0]);
5             int b = Integer.parseInt(args[1]);
6             System.out.println("Result: " + (a / b));
7
8         } catch (ArrayIndexOutOfBoundsException e) {
9             System.out.println("Provide two command line
10                arguments.");
11         } catch (NumberFormatException e) {
12             System.out.println("Arguments must be integers.");
13         } catch (ArithmeticException e) {
14             System.out.println("Division by zero not allowed.")
15             ;
16         } finally {
17             System.out.println("Finally block executed.");
18         }
19     }
20 }
```

## Experiment 14 Input Validation Using throw and throws Keywords

### Objective

To demonstrate the use of throw and throws for user input validation.

### Source Code

```
1 import java.util.Scanner;
2
3 public class Exp14_ThrowThrows {
4     static int readAge(Scanner sc) throws Exception {
5         System.out.print("Enter age: ");
6         int age = sc.nextInt();
7         if(age < 0) throw new Exception("Age cannot be negative
8             !");
9         return age;
10    }
11
12    public static void main(String[] args) {
13        Scanner sc = new Scanner(System.in);
14        try {
15            int age = readAge(sc);
16            System.out.println("Valid Age: " + age);
17        } catch (Exception e) {
18            System.out.println("Error: " + e.getMessage());
19        }
20    }
```

## Experiment 15 Multithreading Implementation Using Runnable Interface

### Objective

To create two threads using the Runnable interface and execute them simultaneously.

### Source Code

```
1  class Task1 implements Runnable {
2      public void run() {
3          for(int i=1;i<=5;i++){
4              System.out.println("Task1: " + i);
5          }
6      }
7  }
8
9  class Task2 implements Runnable {
10     public void run() {
11         for(int i=1;i<=5;i++){
12             System.out.println("Task2: " + i);
13         }
14     }
15 }
16
17 public class Exp15_RunnableThreads {
18     public static void main(String[] args) {
19         Thread t1 = new Thread(new Task1());
20         Thread t2 = new Thread(new Task2());
21         t1.start();
22         t2.start();
23     }
24 }
```

## Experiment 16 Package Creation and Implementation Using User-Defined Package

### Objective

To create a package called Mathematics with a Matrix class that has add and subtract methods, and use it by importing the package.

### Source Code

```
1 // File: Mathematics/Matrix.java
2 package Mathematics;
3
4 public class Matrix {
5     public int [][] add(int [][] a, int [][] b){
6         int [][] c = new int [2][2];
7         for(int i=0;i<2;i++)
8             for(int j=0;j<2;j++)
9                 c[i][j] = a[i][j] + b[i][j];
10        return c;
11    }
12
13    public int [][] sub(int [][] a, int [][] b){
14        int [][] c = new int [2][2];
15        for(int i=0;i<2;i++)
16            for(int j=0;j<2;j++)
17                c[i][j] = a[i][j] - b[i][j];
18        return c;
19    }
20 }
21 // File: Exp16_PackageUse.java
22 import Mathematics.Matrix;
23
24 public class Exp16_PackageUse {
25     public static void main(String[] args) {
26         int [][] a = {{1,2},{3,4}};
27         int [][] b = {{5,6},{7,8}};
28
29         Matrix m = new Matrix();
30         int [][] add = m.add(a,b);
31
32         System.out.println("Addition:");
33         for(int i=0;i<2;i++){
34             for(int j=0;j<2;j++){
35                 System.out.print(add[i][j] + " ");
36             }
37             System.out.println();
38         }
39     }
40 }
```

## Experiment 17 Implementation of Various Input Methods in Java

### Objective

To take input using Command Line Arguments, DataInputStream, BufferedReader, Scanner, and Console methods.

### Source Code

```
1 import java.io.*;
2 import java.util.*;
3
4 public class Exp17_AllInputMethods {
5     public static void main(String[] args) throws Exception {
6
7         // 1) Command Line Arguments
8         if(args.length > 0) {
9             System.out.println("CLA Input: " + args[0]);
10        } else {
11            System.out.println("CLA Input: (none provided)");
12        }
13
14        // 2) DataInputStream
15        DataInputStream dis = new DataInputStream(System.in);
16        System.out.print("DataInputStream name: ");
17        String nameDIS = dis.readLine();
18
19        // 3) BufferedReader
20        BufferedReader br = new BufferedReader(new
21            InputStreamReader(System.in));
22        System.out.print("BufferedReader city: ");
23        String cityBR = br.readLine();
24
25        // 4) Scanner
26        Scanner sc = new Scanner(System.in);
27        System.out.print("Scanner age: ");
28        int age = sc.nextInt();
29        sc.nextLine();
30
31        // 5) Console
32        Console con = System.console();
33        String userConsole = "(Console not available)";
34        if(con != null){
35            userConsole = con.readLine("Console username: ");
36        }
37
38        System.out.println("\n--- Collected Data ---");
39        System.out.println("DIS Name: " + nameDIS);
40        System.out.println("BR City: " + cityBR);
```

```
40         System.out.println("Scanner Age: " + age);
41         System.out.println("Console Username: " + userConsole);
42     }
43 }
```

## Experiment 18 Secure Input Handling Using Console Class

### Objective

To read the username and password using the Console class, with the password hidden while typing, and then display both.

### Source Code

```
1 import java.io.Console;
2
3 public class Exp18_ConsolePassword {
4     public static void main(String[] args) {
5         Console con = System.console();
6         if(con == null){
7             System.out.println("Console not available. Run in
8                 terminal, not in IDE output window.");
9             return;
10        }
11
12        String user = con.readLine("Enter username: ");
13        char[] pass = con.readPassword("Enter password: ");
14
15        System.out.println("Username: " + user);
16        System.out.println("Password: " + String.valueOf(pass))
17        ;
18    }
19 }
```

## Experiment 19 Development of RESTful Employee Management System Using Spring Boot

### Objective

To create a Spring Boot Employee Management System that performs CRUD operations using RESTful APIs.

### Source Code

```
1 // NOTE: Minimal working Spring Boot structure (multiple files
  shown).
2 // File: pom.xml (dependencies snippet)
3 /*
4 spring-boot-starter-web
5 */
6
7 // File: EmployeeApp.java
8 import org.springframework.boot.SpringApplication;
9 import org.springframework.boot.autoconfigure.
   SpringApplication;
10
11 @SpringBootApplication
12 public class EmployeeApp {
13     public static void main(String[] args) {
14         SpringApplication.run(EmployeeApp.class, args);
15     }
16 }
17
18 // File: Employee.java
19 class Employee {
20     public int id;
21     public String name;
22     public String role;
23
24     public Employee() {}
25     public Employee(int id, String name, String role){
26         this.id = id; this.name = name; this.role = role;
27     }
28 }
29
30 // File: EmployeeController.java
31 import java.util.*;
32 import org.springframework.web.bind.annotation.*;
33
34 @RestController
35 @RequestMapping("/api/employees")
36 class EmployeeController {
37     private final Map<Integer, Employee> store = new HashMap
   <>();
```



```
38
39     @PostMapping
40     public Employee create(@RequestBody Employee e){
41         store.put(e.id, e);
42         return e;
43     }
44
45     @GetMapping
46     public Collection<Employee> all(){
47         return store.values();
48     }
49
50     @GetMapping("/{id}")
51     public Employee one(@PathVariable int id){
52         return store.get(id);
53     }
54
55     @PutMapping("/{id}")
56     public Employee update(@PathVariable int id, @RequestBody
57         Employee e){
58         store.put(id, e);
59         return e;
60     }
61
62     @DeleteMapping("/{id}")
63     public String delete(@PathVariable int id){
64         store.remove(id);
65         return "Deleted: " + id;
66     }
67 }
```

## Experiment 20    Enhanced RESTful Employee CRUD Application Using Spring Boot

### Objective

To create a Spring Boot Employee Management System that performs CRUD operations using RESTful APIs (Repeat/Practice).

### Source Code

```
1 // Repeat Experiment 19 with different fields or validation.
2 // Suggestion: Add "email" field + basic validation checks.
3
4 class Employee2 {
5     public int id;
6     public String name;
7     public String email;
8
9     public Employee2() {}
10    public Employee2(int id, String name, String email){
11        this.id=id; this.name=name; this.email=email;
12    }
13 }
```

## Experiment 21 Development of Frontend-Based Employee Management System Using Thymeleaf

### Objective

To create and test a Spring Boot frontend for an Employee Management System using Thymeleaf templates to Add and View Employees.

### Source Code

```
1  /*
2  Dependencies:
3  spring-boot-starter-web
4  spring-boot-starter-thymeleaf
5  */
6
7  // File: EmployeeFrontController.java
8  import java.util.*;
9  import org.springframework.stereotype.Controller;
10 import org.springframework.ui.Model;
11 import org.springframework.web.bind.annotation.*;
12
13 @Controller
14 class EmployeeFrontController {
15     private final List<Employee> list = new ArrayList<>();
16
17     @GetMapping("/")
18     public String home(Model model){
19         model.addAttribute("employees", list);
20         model.addAttribute("employee", new Employee());
21         return "index";
22     }
23
24     @PostMapping("/add")
25     public String add(@ModelAttribute Employee employee){
26         list.add(employee);
27         return "redirect:/";
28     }
29 }
30
31 /*
32 File: templates/index.html (Thymeleaf)
33 <form action="/add" method="post">
34     <input name="id" placeholder="ID"/>
35     <input name="name" placeholder="Name"/>
36     <input name="role" placeholder="Role"/>
37     <button type="submit">Add</button>
38 </form>
39
40 <ul>
```

```
41     <li th:each="e : ${employees}">
42         <span th:text="${e.id}"></span> -
43         <span th:text="${e.name}"></span> -
44         <span th:text="${e.role}"></span>
45     </li>
46 </ul>
47 */
```

## Experiment 22 Implementation of In-Memory CRUD Operations Using REST API

### Objective

To implement CRUD operations for an Employee Management System using a REST API with ArrayList/Map (in-memory storage).

### Source Code

```
1 import java.util.*;
2 import org.springframework.web.bind.annotation.*;
3
4 @RestController
5 @RequestMapping("/api/v2/employees")
6 class Exp22_InMemoryCRUD {
7     private final List<Employee> list = new ArrayList<>();
8
9     @PostMapping
10    public Employee add(@RequestBody Employee e){
11        list.add(e);
12        return e;
13    }
14
15    @GetMapping
16    public List<Employee> getAll(){
17        return list;
18    }
19
20    @PutMapping("/{id}")
21    public String update(@PathVariable int id, @RequestBody
22        Employee e){
23        for(int i=0;i<list.size();i++){
24            if(list.get(i).id == id){
25                list.set(i, e);
26                return "Updated";
27            }
28        }
29        return "Not Found";
30    }
31
32    @DeleteMapping("/{id}")
33    public String remove(@PathVariable int id){
34        return list.removeIf(x -> x.id == id) ? "Deleted" : "
35        Not Found";
36    }
37 }
```

## Experiment 23 Book Store Management System Using Spring Boot and In-Memory Storage

### Objective

To develop a Spring Boot Book Store Management System (in-memory ArrayList) with CRUD operations and a Bootstrap-based web interface.

### Source Code

```
1  /*
2  Dependencies:
3  spring-boot-starter-web
4  spring-boot-starter-thymeleaf
5  */
6
7  // File: Book.java
8  class Book {
9      public int id;
10     public String title;
11     public double price;
12
13     public Book() {}
14     public Book(int id, String title, double price){
15         this.id=id; this.title=title; this.price=price;
16     }
17 }
18
19 // File: BookController.java
20 import java.util.*;
21 import org.springframework.stereotype.Controller;
22 import org.springframework.ui.Model;
23 import org.springframework.web.bind.annotation.*;
24
25 @Controller
26 class BookController {
27     private final List<Book> books = new ArrayList<>();
28
29     @GetMapping("/books")
30     public String view(Model model){
31         model.addAttribute("books", books);
32         model.addAttribute("book", new Book());
33         return "books";
34     }
35
36     @PostMapping("/books/add")
37     public String add(@ModelAttribute Book b){
38         books.add(b);
39         return "redirect:/books";
```

```
40     }
41
42     @PostMapping("/books/updatePrice")
43     public String updatePrice(@RequestParam int id,
44                               @RequestParam double price){
45         for(Book b: books){
46             if(b.id == id){ b.price = price; break; }
47         }
48         return "redirect:/books";
49     }
50
51     @PostMapping("/books/delete")
52     public String delete(@RequestParam int id){
53         books.removeIf(b -> b.id == id);
54         return "redirect:/books";
55     }
56 }
57
58 /*
59 File: templates/books.html (Bootstrap idea)
60 - Form to Add Book
61 - Table to View Books
62 - Form to Update Price by ID
63 - Form to Delete by ID
64 */
```

## Experiment 24 Book Store Management System Using Spring Boot, H2 Database, and JPA

### Objective

To develop a Spring Boot Book Store Management System using the H2 Database and Spring Data JPA, performing CRUD operations via the Thymeleaf UI.

### Source Code

```
1  /*
2  Dependencies:
3  spring-boot-starter-web
4  spring-boot-starter-thymeleaf
5  spring-boot-starter-data-jpa
6  com.h2database:h2
7  */
8
9  // File: BookEntity.java
10 import jakarta.persistence.*;
11
12 @Entity
13 class BookEntity {
14     @Id
15     private int id;
16     private String title;
17     private double price;
18
19     public BookEntity() {}
20     public BookEntity(int id, String title, double price){
21         this.id=id; this.title=title; this.price=price;
22     }
23     public int getId(){ return id; }
24     public void setId(int id){ this.id=id; }
25     public String getTitle(){ return title; }
26     public void setTitle(String title){ this.title=title; }
27     public double getPrice(){ return price; }
28     public void setPrice(double price){ this.price=price; }
29 }
30
31 // File: BookRepo.java
32 import org.springframework.data.jpa.repository.JpaRepository;
33
34 interface BookRepo extends JpaRepository<BookEntity, Integer>
35     {}
36
37 // File: BookUIController.java
38 import org.springframework.stereotype.Controller;
39 import org.springframework.ui.Model;
```



```
39 import org.springframework.web.bind.annotation.*;
40
41 @Controller
42 class BookUIController {
43     private final BookRepo repo;
44
45     BookUIController(BookRepo repo){ this.repo = repo; }
46
47     @GetMapping("/store")
48     public String store(Model model){
49         model.addAttribute("books", repo.findAll());
50         model.addAttribute("book", new BookEntity());
51         return "store";
52     }
53
54     @PostMapping("/store/add")
55     public String add(@ModelAttribute BookEntity b){
56         repo.save(b);
57         return "redirect:/store";
58     }
59
60     @PostMapping("/store/updatePrice")
61     public String update(@RequestParam int id, @RequestParam
62         double price){
63         BookEntity b = repo.findById(id).orElse(null);
64         if(b != null){
65             b.setPrice(price);
66             repo.save(b);
67         }
68         return "redirect:/store";
69     }
70
71     @PostMapping("/store/delete")
72     public String delete(@RequestParam int id){
73         repo.deleteById(id);
74         return "redirect:/store";
75     }
76
77     /*
78     File: application.properties
79     spring.h2.console.enabled=true
80     spring.datasource.url=jdbc:h2:mem:testdb
81     spring.jpa.hibernate.ddl-auto=update
82
83     File: templates/store.html
84     - Same as books.html but bound to JPA repository data
85     */
```